Машинное обучение · Лекция 9

От линейной модели к нейросети

Начало второй половины курса: глубокое обучение

Рустам Азимов

Где мы находимся?

Начинаем глубокое обучение (Deep Learning): нейросети учатся выделять признаки из изображений, текста и звука.

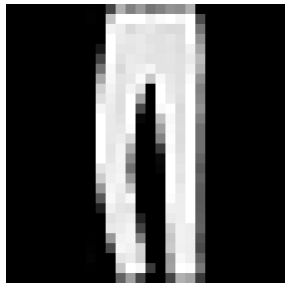
Переобучение, валидация и утечки данных остаются важны.

Сквозной пример: распознаём одежду

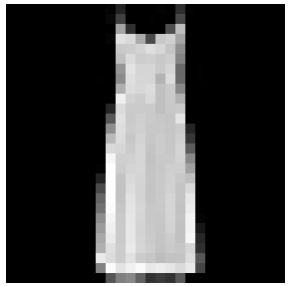
Сможет ли сеть отличить футболку от ботинка?



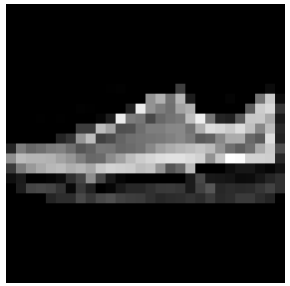
Футболка



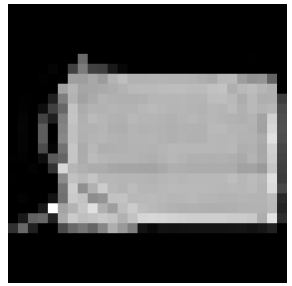
Брюки



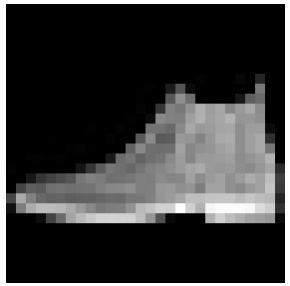
Платье



Кроссовок



Сумка



Ботинок

Вход: изображение 28 × 28. Ответ: один из 10 классов.

На практике сеть должна узнать категорию новой вещи.

Что видит сеть вместо картинки

Для компьютера изображение — таблица чисел



Одна футболка: 784 пикселя

235	226	198	190
204	184	190	192
195	197	194	193
199	197	195	197

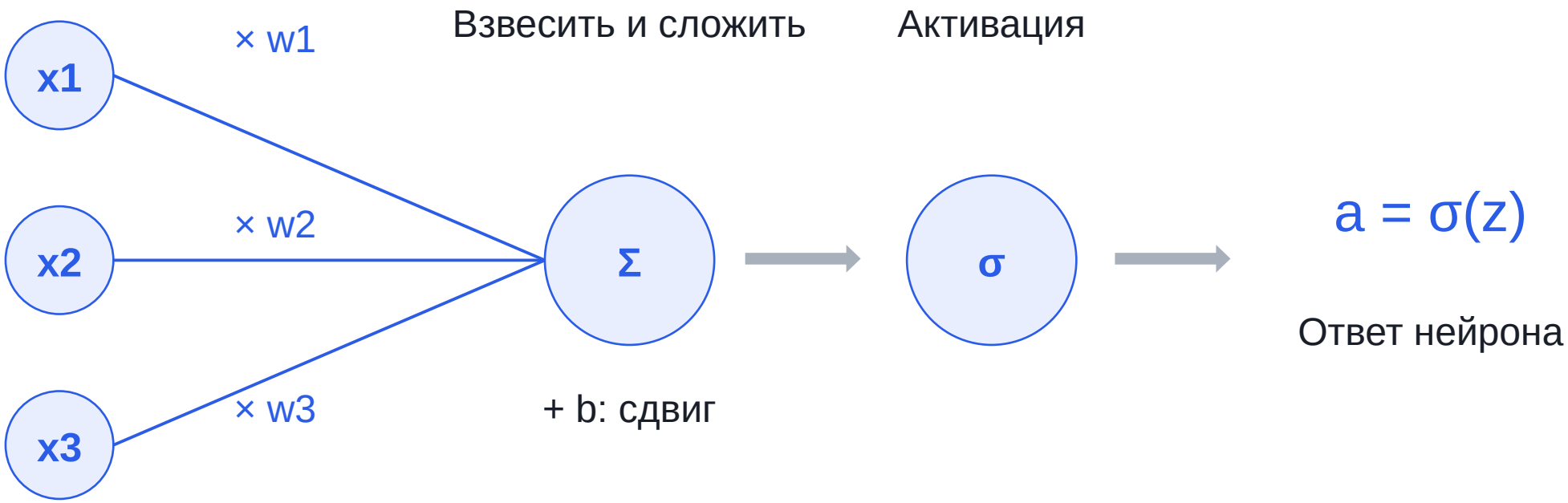
Настоящий фрагмент 4 × 4 из этой картинки

0 = чёрный, 255 = белый.

Делим на 255: например, 235 / 255 ≈ 0.92.

Устройство искусственного нейрона

Каждый вход усиливает или ослабляет общий сигнал



Классический перцептрон использует пороговую активацию. В примере с вероятностью используем сигмоиду.

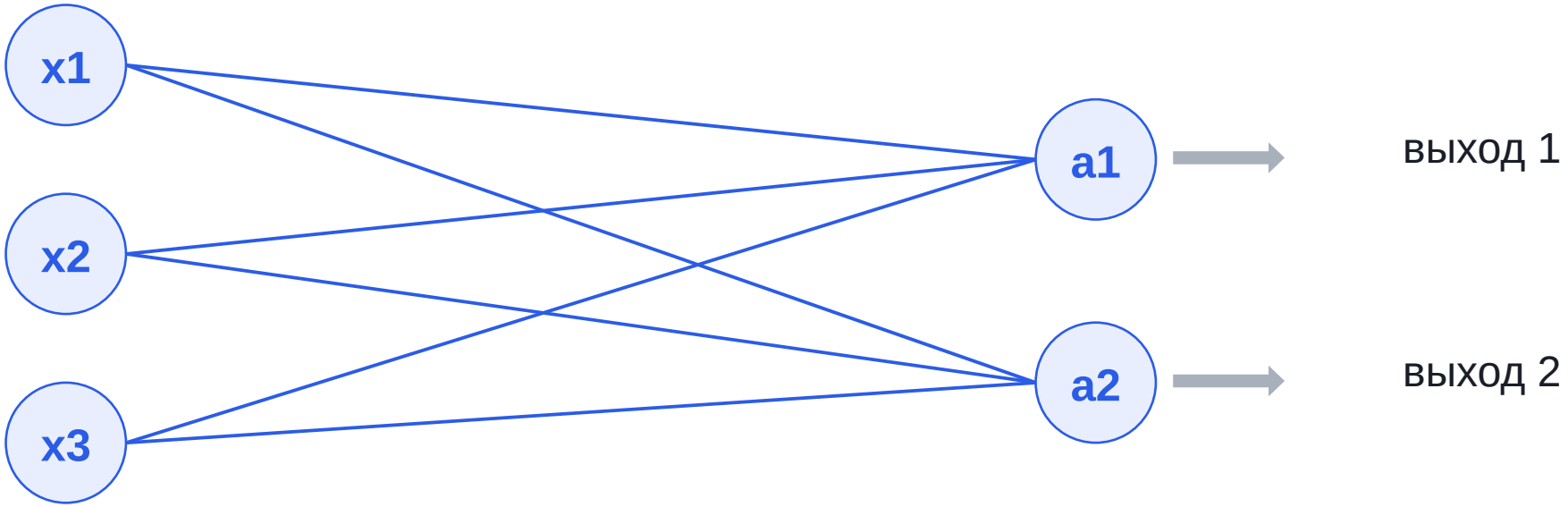
Однослойная сеть: несколько нейронов параллельно

Одни входы, разные веса, разные ответы

Входные признаки

Каждая связь — вес

Один слой нейронов



Один слой весов, без скрытых слоёв. Входы отдельно слоем не считаем.

Что осталось позади и что впереди?

Первая половина

— Таблицы: строки, столбцы, признаки руками

— Модель выбирают под данные

— Бустинг как рабочая лошадка

— Признаки конструирует человек (Feature Engineering)

Вторая половина

— Текст, изображения, звук — сырые данные

— Одна архитектура на множество задач

— Трансформер как рабочая лошадка

— **Модель учит представления (Representation Learning)**

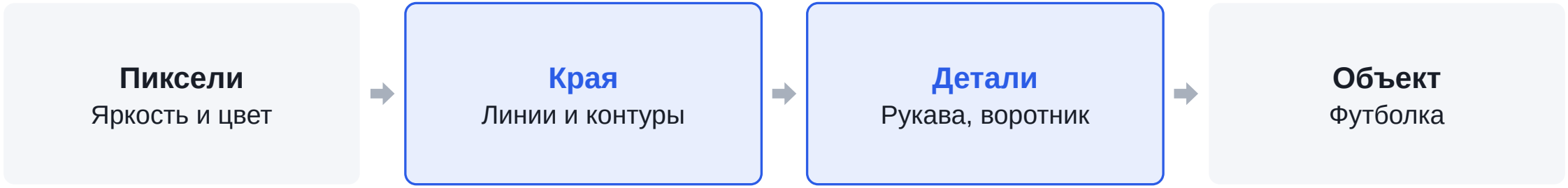
Почему нейросети стали полезны

Идея старая. Для хорошего результата понадобились данные и вычисления

- На нескольких фотографиях трудно выучить, как выглядят разные виды одежды
- Большие наборы изображений дают много примеров каждого класса
- Видеокарта (GPU) быстро выполняет много похожих вычислений
- Удачные активации и способы обучения помогают обучать глубокие сети

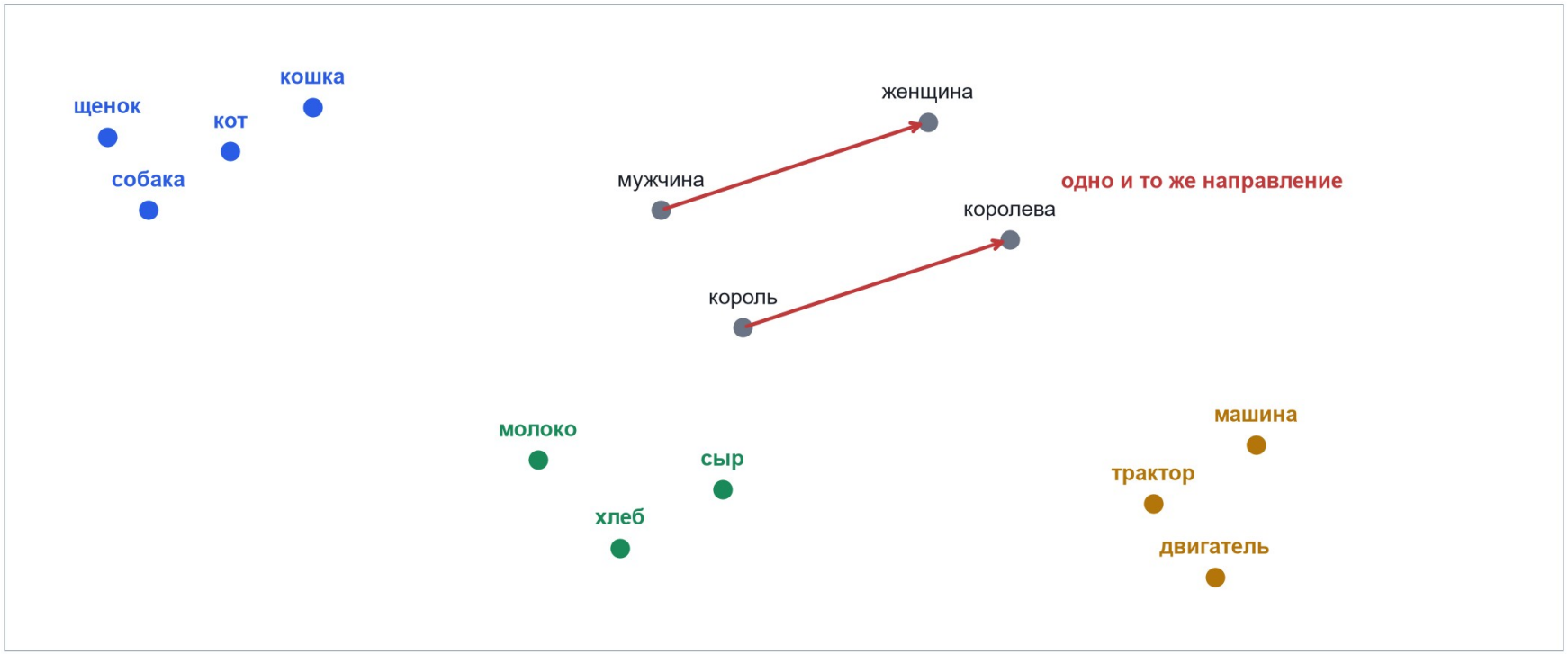
Как сеть учит представления

Representation Learning: каждый слой преобразует предыдущие признаки



Упрощённая интуиция для сети изображений: ранние слои находят простые закономерности, поздние комбинируют их. Мы задаём задачу, а веса признаков сеть учит по данным.

Embeddings: похожие объекты рядом



два измерения из нескольких сотен

Представление — вектор чисел. Близкие по смыслу слова могут получать близкие векторы. Здесь условная схема в двух измерениях, а не измеренные результаты модели.

Где глубокое обучение выигрывает, а где нет?

Когда нейросеть полезна

— Изображения, видео, звук

— Текст и всё, что связано с языком

— Задачи, где данных много, а разметка дешёвая или её нет

— Перенос знаний предобученной модели (Transfer Learning)

Когда начать с классики

— Таблицы среднего размера — там бустинг

— Данных мало, признаков десяток

— Нужна объяснимость каждого решения

— Нужна небольшая модель, ресурсов мало

Проверим интуицию

У вас таблица на 20 тысяч строк и 40 признаков, задача — предсказать отток. Стоит ли брать нейросеть?

А Да, нейросети сейчас всегда лучше

Б Нет, на таких данных бустинг обычно сильнее и дешевле

В Да, если сделать её достаточно глубокой

Г Зависит только от наличия видеокарты

Ответьте про себя, прежде чем перевернуть слайд

Проверим интуицию — разбор

У вас таблица на 20 тысяч строк и 40 признаков, задача — предсказать отток. Стоит ли брать нейросеть?

А Да, нейросети сейчас всегда лучше

Б Нет, на таких данных бустинг обычно сильнее и дешевле — верно

В Да, если сделать её достаточно глубокой

Г Зависит только от наличия видеокарты

Для такой таблицы начните с бустинга: это сильный и удобный базовый вариант. Качество всё равно сравниваем на одинаковой валидации. Нейросети особенно полезны для изображений, текста и звука, где трудно вручную описать все нужные признаки.

1

Из чего собрана сеть

Перцептрон (Perceptron), слои, активации

Один нейрон — это уже знакомая формула

Нейрон берёт вход, взвешивает и добавляет сдвиг — ровно как линейная модель:

$$z = w^\top x + b$$

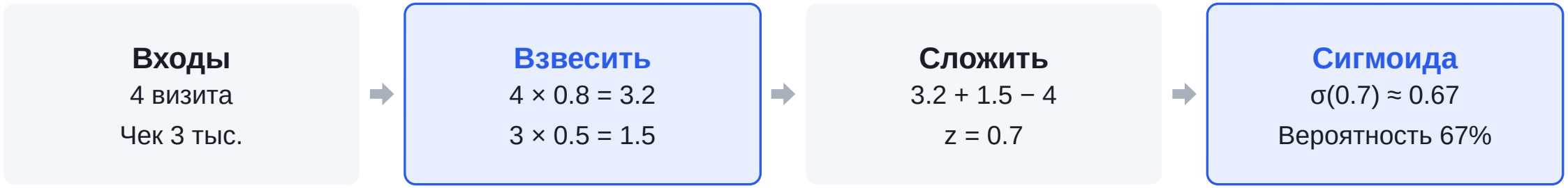
А потом пропускает результат через нелинейность:

$$a = \sigma(z)$$

Логистическая регрессия — это в точности один нейрон с сигмоидой. Всё остальное в этой лекции — как из таких нейронов собрать что-то большее

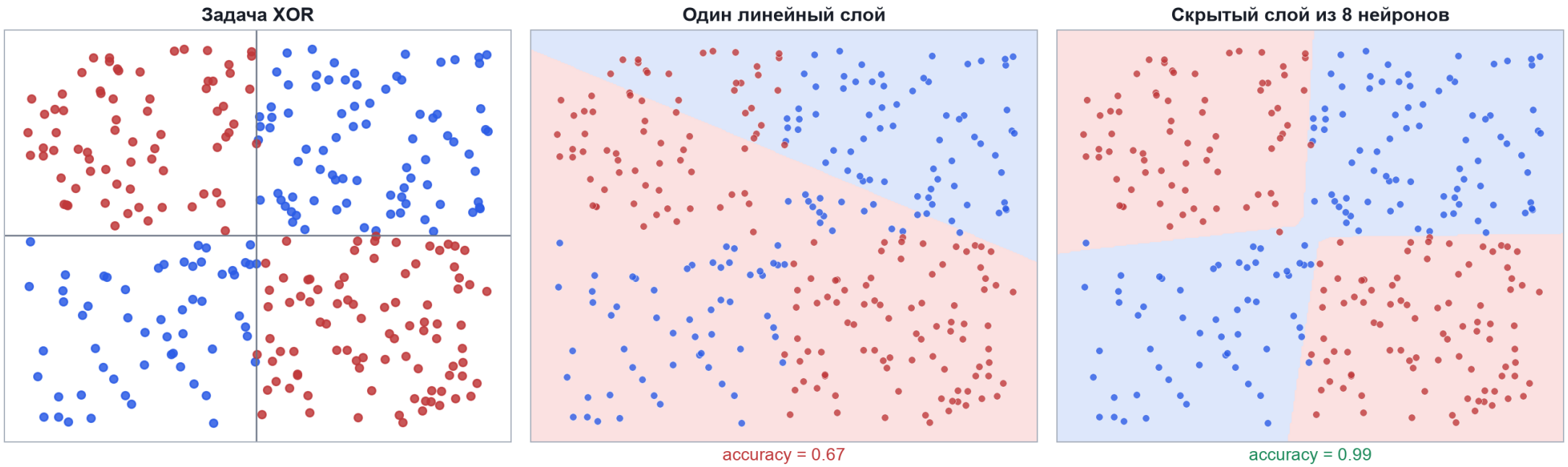
Один нейрон на числах: купит ли клиент?

Веса показывают, как признаки влияют на общий сигнал



Ещё один визит прибавит к z число 0.8: σ(1.5) ≈ 82%. Положительный вес усиливает сигнал, отрицательный ослабляет. Сдвиг b = −4 задаёт исходный уровень.

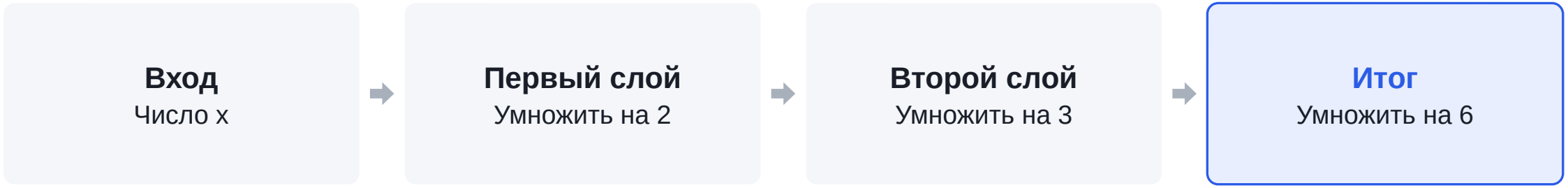
Почему без нелинейности ничего не выйдет?



Посмотрите на цвета точек: одной прямой разделить их не получается. Сеть со скрытым слоем и нелинейной активацией может изгибать границу. Здесь показан результат одного учебного эксперимента.

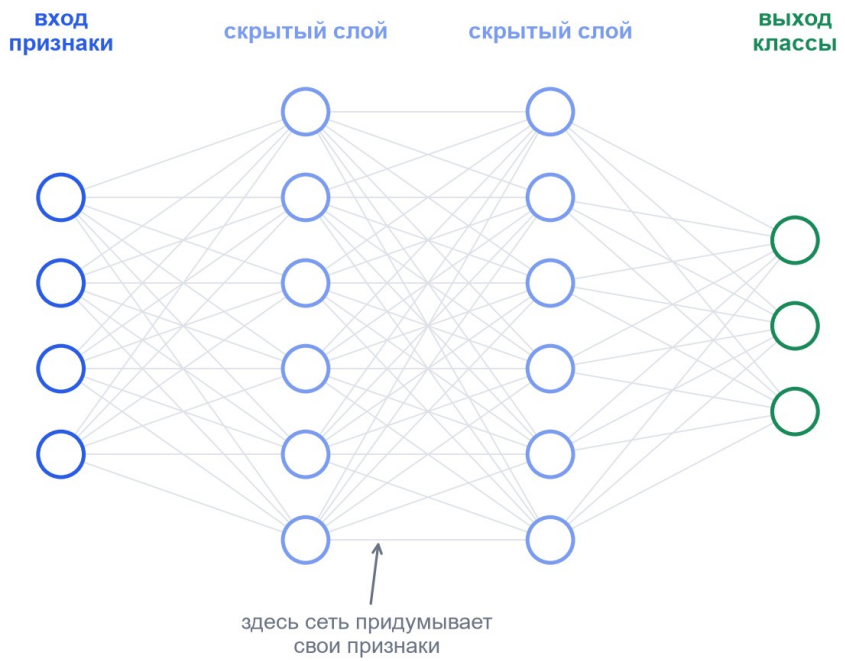
Почему одних линейных слоёв мало

Сначала на простых числах, затем тот же принцип для матриц



Два линейных преобразования можно заменить одним. Добавление сдвигов этого не меняет. Активация между слоями позволяет получить более сложное правило.

Многослойный перцептрон (MLP)



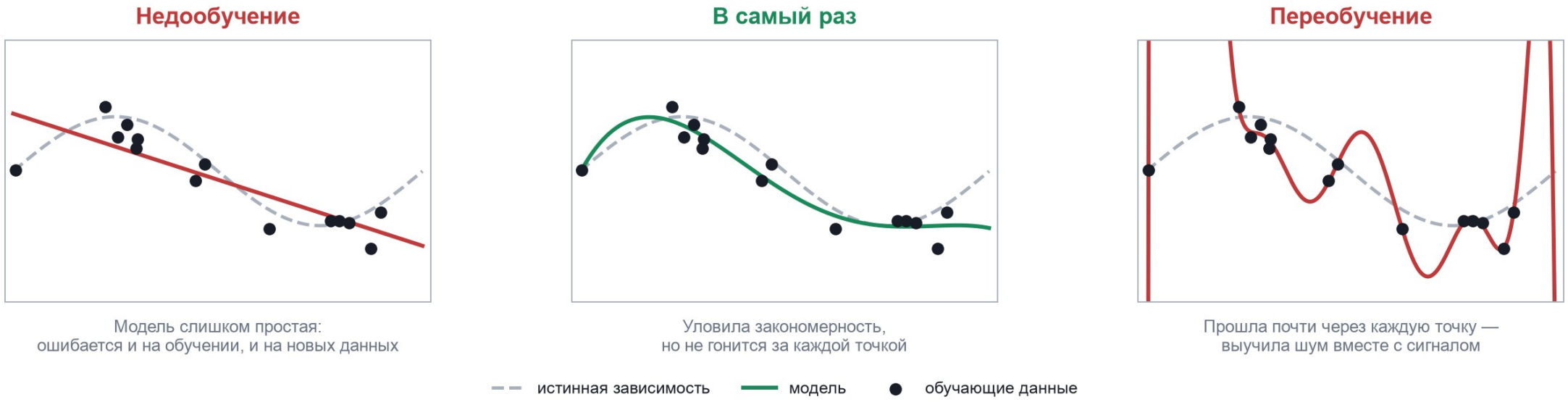
Слева входные признаки, справа выход. Каждая стрелка — вес, каждый круг — нейрон со своей нелинейностью. Скрытый слой (Hidden Layer) учит промежуточное представление данных. Это и есть Representation Learning.

Сколько чисел сеть должна выучить?

Параметры — веса связей и сдвиги нейронов

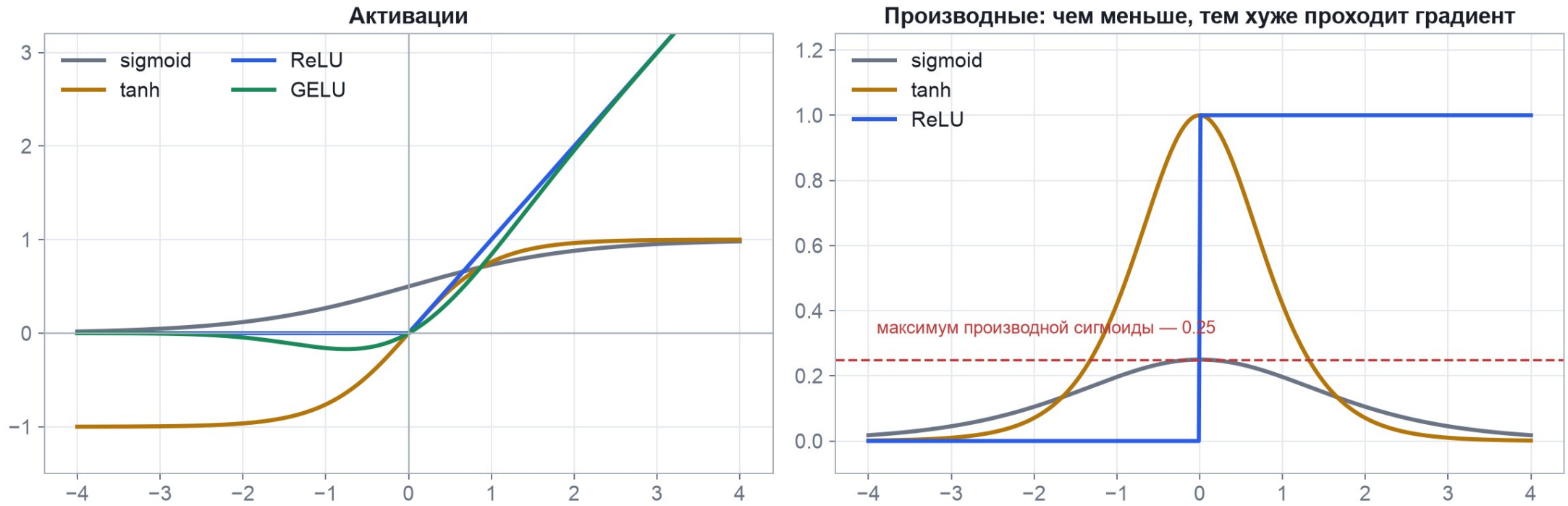
Слой	Размерности	Параметров
Вход → скрытый	784 → 256	784 · 256 + 256 = 200 960
Скрытый → скрытый	256 → 256	256 · 256 + 256 = 65 792
Скрытый → выход	256 → 10	256 · 10 + 10 = 2 570
Итого	небольшая сеть для картинок 28×28	269 322

Сеть может запомнить примеры и ошибаться на новых



Как студент, который выучил ответы на билеты: знакомые задачи решает, новые — хуже. Поэтому проверяем сеть на отдельной валидационной выборке, которую она не видела при обучении.

Функции активации



Горизонтальная ось — вход нейрона, вертикальная — выход. Сигмоида плавно сжимает ответ в диапазон 0–1. ReLU отрезает отрицательную часть. Для нашей первой сети используем ReLU в скрытых слоях.

Для начала достаточно двух активаций

GELU, SiLU и другие варианты разберём позже

Активация	Простое правило	Где используем
ReLU	Отрицательное → 0, положительное оставляем	В скрытых слоях нашего MLP
Sigmoid	Любое число → число от 0 до 1	Выход для вероятности одного события

ReLU: простое правило внутри нейрона

ReLU читается как «рэлу»

z = -2	→	ReLU = max(0, z)	→	**a = 0**
z = 0	→	ReLU = max(0, z)	→	**a = 0**
z = 3	→	ReLU = max(0, z)	→	**a = 3**

Отрицательные значения обнуляем. Положительные пропускаем. Из-за этого правила сеть может строить нелинейную границу.

Проверим интуицию

Сеть из пяти слоёв, между слоями активаций нет. Что она может выучить?

А Любую сложную зависимость — слоёв же пять

Б Ровно то же, что одна линейная модель

В Ничего, обучение разойдётся

Г Зависит от числа нейронов в слоях

Ответьте про себя, прежде чем перевернуть слайд

Проверим интуицию — разбор

Сеть из пяти слоёв, между слоями активаций нет. Что она может выучить?

А Любую сложную зависимость — слоёв же пять

Б Ровно то же, что одна линейная модель — верно

В Ничего, обучение разойдётся

Г Зависит от числа нейронов в слоях

Композиция линейных преобразований — снова линейное преобразование. Пять матриц перемножатся в одну, и сеть окажется обычной линейной моделью с лишними параметрами: обучаться будет дольше, а выучит то же самое. Именно нелинейность между слоями делает сеть сетью.

Сколько брать слоёв и нейронов?

Для практики возьмём готовый небольшой MLP и будем менять по одному параметру

С чего начать

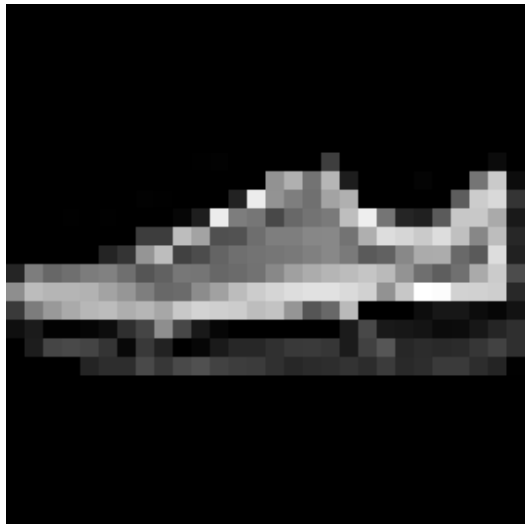
- Два-три скрытых слоя для табличных задач
- Ширина 128–512 — разумный старт
- Одинаковая ширина слоёв проще в настройке, чем сужающаяся

Как двигаться дальше

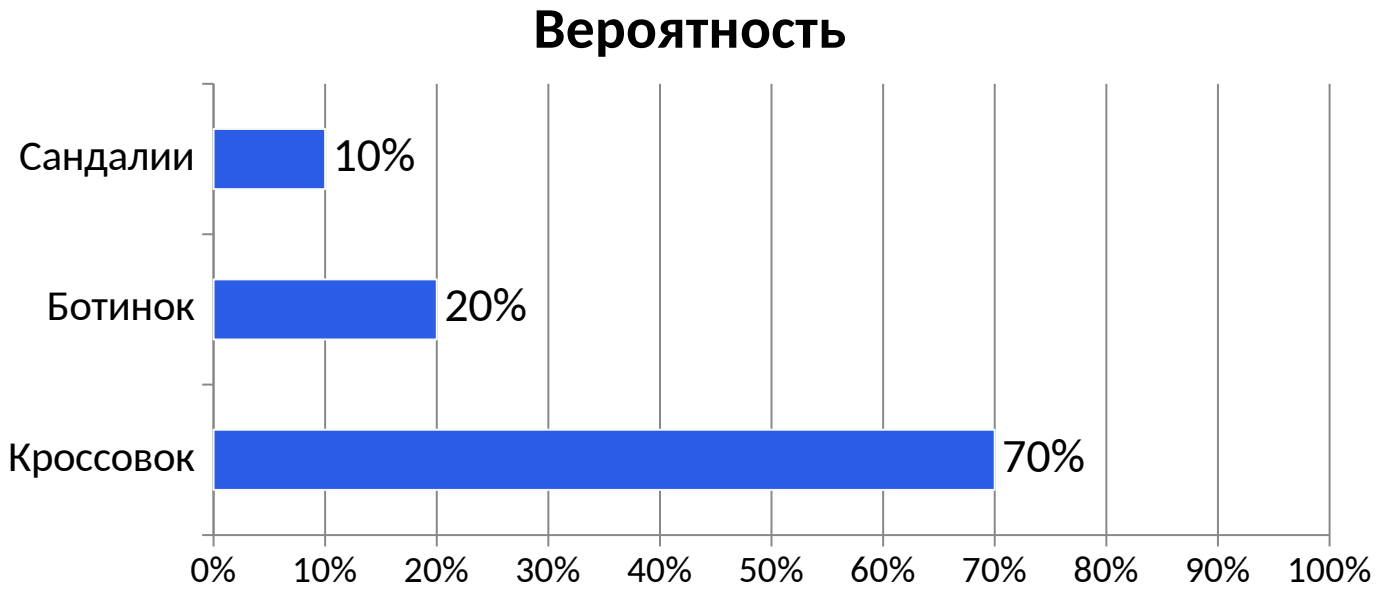
- Сначала проверьте, что сеть может выучить маленькую выборку
- Если на обучении хорошо, а на валидации хуже — боремся с переобучением
- Не меняйте всё сразу: один параметр, один эксперимент

Как сеть выбирает класс

Учебный пример: оставим только три вида обуви



Что изображено?



Выбираем наибольшую вероятность: кроссовок.

Вероятности придуманы для объяснения. Это не результат обученной модели.

Выход для классификации: softmax

Последний слой выдаёт оценки классов (Logits). Softmax превращает их в вероятности:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

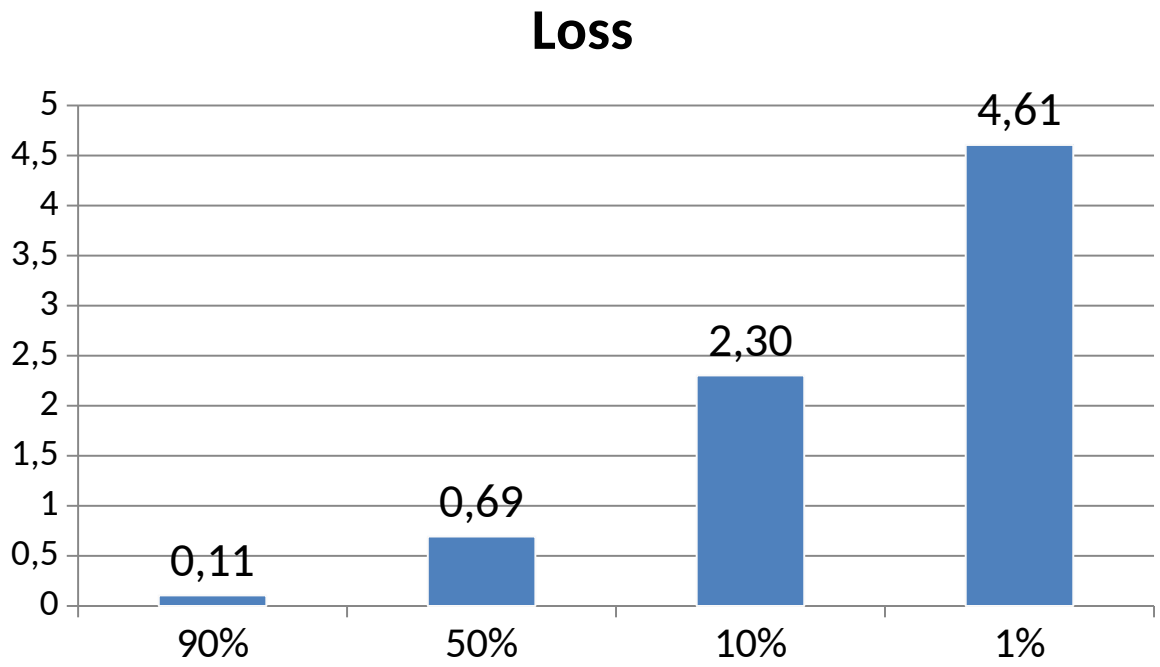
Функция потерь (Loss Function) здесь — перекрёстная энтропия (Cross-Entropy):

$$L = -\sum_{k=1}^{K} y_k \log p_k$$

Для правильного класса это просто минус логарифм предсказанной вероятности: уверенная ошибка штрафуется жёстко, неуверенная — мягко

Loss: насколько сеть ошиблась

Cross-Entropy сильно штрафует низкую вероятность правильного ответа



На фото кроссовок. Смотрим вероятность именно кроссовка.

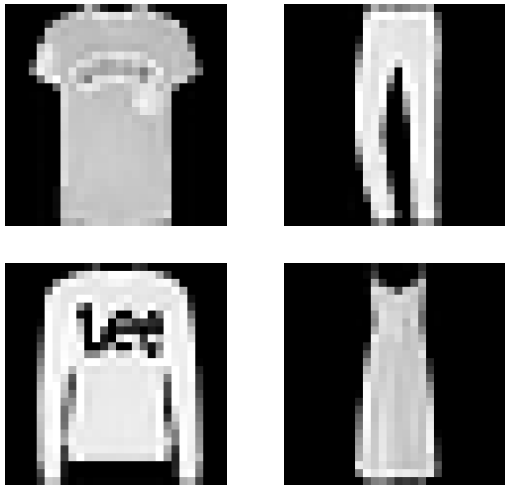
Дали ему 90%: малый штраф.
Дали только 1%: большой штраф.

По горизонтали: вероятность правильного класса. По вертикали: Cross-Entropy.

Батч, шаг и эпоха на 12 картинках

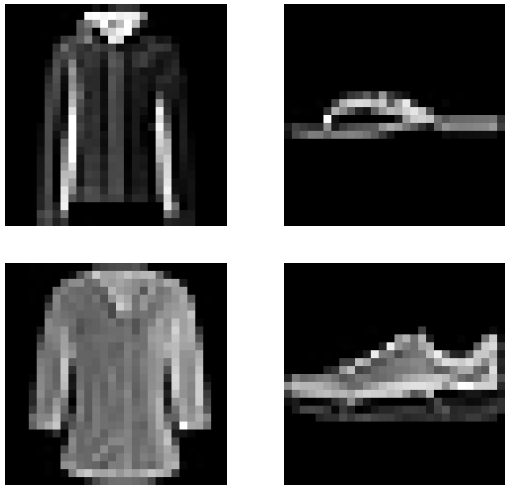
Batch = порция примеров. Step = обновление весов. Epoch = полный проход

Батч 1: 4 картинки



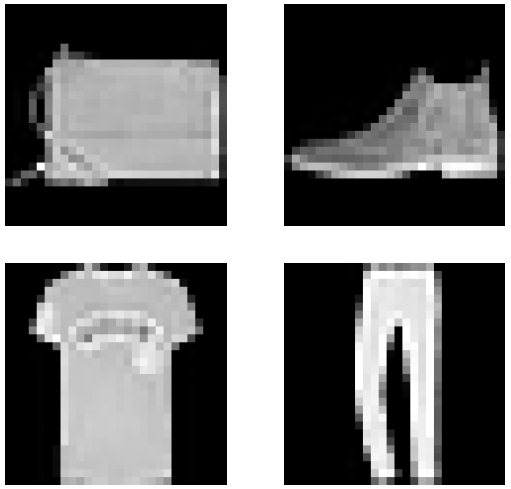
Обновление весов №1

Батч 2: 4 картинки



Обновление весов №2

Батч 3: 4 картинки



Обновление весов №3

12 картинок / 4 в батче = 3 шага. Посмотрели все 12 картинок один раз — прошла 1 эпоха.

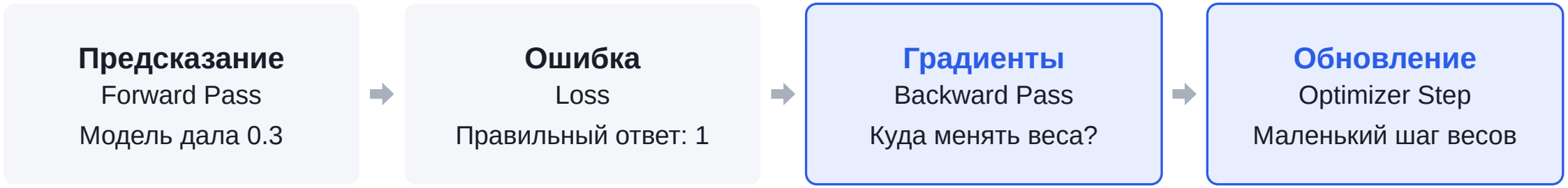
2

Как сеть учится?

Вычислительный граф и backpropagation

Обучение: поправляем веса по ошибке

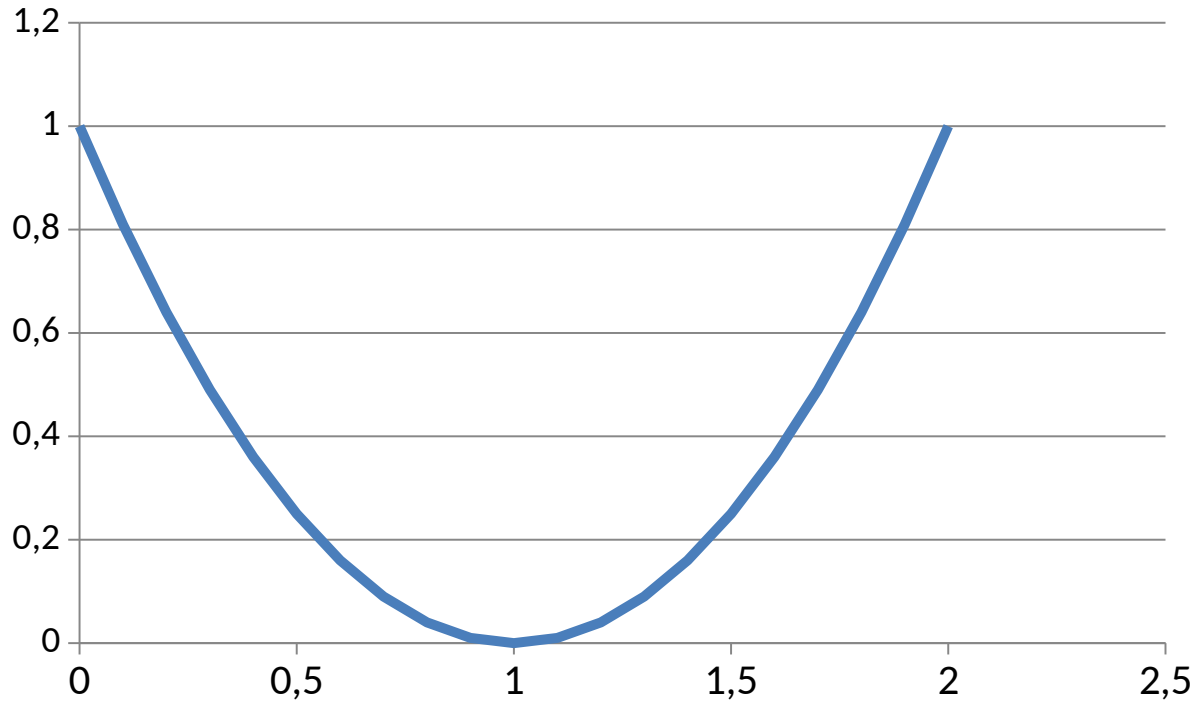
Сначала ответ модели, затем изменение параметров



Повторяем на следующем батче. Градиент показывает локальное направление изменения потерь. Оптимизатор меняет веса так, чтобы уменьшить потери на текущем батче при достаточно малом шаге.

Градиент подсказывает, куда менять вес

Для понимания возьмём одну ручку настройки: $L = (w - 1)^2$



Простой пример с одним весом

Вес $w = 0$, потери $L = 1$.

Градиент равен -2.

Шаг: $w \leftarrow 0 - 0.1 \times (-2)$.

Новый вес 0.2, потери 0.64.

По горизонтали вес w, по вертикали потери L. Идём против градиента: в этом примере вес растёт, а потери падают.

Вычислительный граф



Слева направо получаем ответ и считаем потери: Forward Pass. Справа налево вычисляем, как каждый вес влияет на потери: Backward Pass. Затем оптимизатор делает небольшой шаг весов. Обратный проход сам веса не меняет.

Backpropagation: как ошибка доходит до ранних слоёв

Правило дифференцирования сложной функции, применённое по цепочке слоёв:

$$\frac{\partial L}{\partial w^{(1)}} = \frac{\partial L}{\partial a^{(3)}} \cdot \frac{\partial a^{(3)}}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial w^{(1)}}$$

Каждый множитель показывает чувствительность одного звена. Для этого примера с последовательной цепочкой перемножаем их. Если путей несколько, их вклады складываются.

Почему ранние слои иногда почти не учатся

Vanishing Gradient: сигнал уменьшается на каждом звене



После трёх умножений сигнал стал в 64 раза меньше. Через длинную цепочку он может почти исчезнуть. Обратная проблема — Exploding Gradient: произведение сильно растёт.

Проверим интуицию

Сеть из 30 слоёв с сигмоидами почти не обучается: первые слои стоят на месте. В чём причина?

А Слишком мало данных

Б Производная сигмоиды не больше 0.25, и произведение тридцати таких множителей близко к нулю

В Скорость обучения слишком мала

Г Нужно больше нейронов в каждом слое

Ответьте про себя, прежде чем перевернуть слайд

Проверим интуицию — разбор

Сеть из 30 слоёв с сигмоидами почти не обучается: первые слои стоят на месте. В чём причина?

А Слишком мало данных

Б Производная сигмоиды не больше 0.25, и произведение тридцати таких множителей близко к нулю — верно

В Скорость обучения слишком мала

Г Нужно больше нейронов в каждом слое

Максимум производной сигмоиды — 0.25. Градиент до первого слоя доходит умноженным на три десятка таких множителей, то есть фактически обнуляется, и веса не двигаются. ReLU, нормализация и остаточные связи помогают обучать глубокие сети, но не гарантируют отсутствие проблем с градиентами.

Autograd считает производные за нас

Automatic Differentiation в PyTorch

- Мы записываем вычисления: вход → сеть → потери
- PyTorch запоминает операции, из которых получились потери
- `loss.backward()` вычисляет градиенты и записывает их в `.grad`
- `optimizer.step()` использует градиенты и меняет веса

3

PyTorch

Инструмент, которым будем пользоваться до конца курса

Почему PyTorch?

Что это

- Библиотека для тензорных вычислений на видеокарте
- Автоматическое дифференцирование из коробки
- На нём будем писать и запускать примеры курса

Что даёт

- Код выглядит как обычный Python, отладка обычным отладчиком
- Готовые слои, оптимизаторы, загрузчики данных
- Экосистема: `torchvision`, `timm`, `transformers`

Тензор — таблица чисел для сети

```
import torch

x = torch.tensor([[1., 2.], [3., 4.]])
print(x.shape, x.dtype)          # torch.Size([2, 2]) torch.float32

# Всё привычное на месте
x.mean(dim=0)                    # среднее по столбцам
x @ x.T                          # матричное умножение

# Отличие первое: тензор можно положить на видеокарту
device = "cuda" if torch.cuda.is_available() else "cpu"
x = x.to(device)

# Отличие второе: попросим следить за градиентом
w = torch.tensor([2.0], requires_grad=True)
loss = (w ** 2 + 3 * w).sum()
loss.backward()
print(w.grad)                    # tensor([7.]) — это 2w + 3 при w = 2
```

Сеть как класс

```
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, n_features, n_classes, hidden=256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_features, hidden),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(hidden, hidden),
            nn.ReLU(),
            nn.Linear(hidden, n_classes),   # логиты, без softmax
        )

    def forward(self, x):
        return self.net(x)

model = MLP(n_features=784, n_classes=10).to(device)
print(sum(p.numel() for p in model.parameters()), "параметров")
```

Цикл обучения целиком

```
from torch.utils.data import DataLoader

loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4)

for epoch in range(n_epochs):
    model.train()                              # включаем dropout
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)

        optimizer.zero_grad()                  # 1. обнулили градиенты
        logits = model(xb)                     # 2. прямой проход
        loss = criterion(logits, yb)           # 3. посчитали потери
        loss.backward()                        # 4. обратный проход
        optimizer.step()                       # 5. шаг оптимизатора

    model.eval()                               # выключаем dropout
    with torch.no_grad():                      # градиенты не нужны
        evaluate(model, val_loader)
```

Оценка качества после эпохи

```
@torch.no_grad()
def evaluate(model, loader):
    model.eval()
    correct = total = 0
    loss_sum = 0.0
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        logits = model(xb)
        loss_sum += criterion(logits, yb).item() * len(yb)
        correct += (logits.argmax(dim=1) == yb).sum().item()
        total += len(yb)
    return loss_sum / total, correct / total

val_loss, val_acc = evaluate(model, val_loader)
print(f"эпоха {epoch}: потери {val_loss:.4f}, точность {val_acc:.3f}")
```

Три строки, которые легко забыть

Строка	Что делает	Если забыть
optimizer.zero_grad()	Обнуляет старые градиенты	Градиенты начнут складываться между батчами
model.eval()	Включает режим проверки	Dropout продолжит случайно выключать нейроны
torch.no_grad()	Не строит граф для градиентов	Проверка потратит лишнюю память

4

Батчи и видеокарта

Почему обучение устроено именно так

Что видеокарта делает хорошо?

- Тысячи простых ядер вместо десятка сложных — это машина для матричного умножения
- Обучение сети почти целиком состоит из умножения матриц, поэтому выигрыш достигает десятков раз
- **Условия выигрыша:** данных на устройстве много, а ветвлений в коде мало
  - Отсюда правило: любой цикл по объектам в Python — потенциальное узкое место. Считайте батчами

Зачем нужны батчи?

Размер побольше

— Видеокарта загружена, шаг эффективнее

— Оценка градиента менее шумная

— Упирается в память: не влезет — упадёт

— Большой батч может потребовать другой настройки обучения

Размер поменьше

— Шум в градиенте работает как регуляризация

— Требует меньше памяти

— Обучение дольше по времени

— Для BatchNorm совсем маленькая порция может мешать

Практические ориентиры

- Начинайте с 32–128 объектов в батче и увеличивайте, пока хватает памяти
- После изменения батча снова проверьте качество: скорость обучения может потребовать настройки
- Не влезает в память? Накопление градиента (Gradient Accumulation) позволяет реже обновлять веса без хранения большого батча — разберём в следующей лекции
  - Размер батча — не мелочь настройки, а параметр, который меняет и скорость, и итоговое качество

5

Итоги

Что нужно унести с лекции?

- Нейрон — это линейная модель с нелинейностью; сеть — их композиция
- **Без нелинейности глубина бессмысленна:** слои схлопываются в один
- Обучение — тот же градиентный спуск, а обратное распространение лишь эффективно считает градиент по цепочке
- Произведение множителей вдоль цепочки объясняет и затухание, и взрыв градиента
- Цикл обучения в PyTorch — пять строк, и три из них забывают чаще всего
- Глубокое обучение — инструмент для сырых данных, а не замена бустингу на таблицах

Что дальше?

На практике

— Тензоры и autograd руками

— Свой MLP на FashionMNIST

— Полный цикл обучения с нуля

На следующей лекции

— Инициализация и нормализация

— AdamW, warmup, cosine

— Что делать, когда сеть не учится